Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	13 July 2026
Team ID	SWUID2026196344
Project Name	**Nutrition Assistant**
Maximum Marks	4 Marks

Technical Architecture:

The Technical Architecture of the Nutrition Assistant project illustrates the overall system structure and the interaction between the frontend, backend, database, and users. It provides a clear understanding of how different components work together to deliver secure, efficient, and personalized nutrition management services.

The technical architecture diagram demonstrates the flow of requests and responses between the React.js frontend, Express.js/Node.js backend, and MongoDB database, along with JWT-based authentication and REST API communication.

The deliverables include the Technical Architecture Diagram and the corresponding architectural details describing each system component and its functionality.

Project: Nutrition Assistant – MERN Stack Web Application

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/

Guidelines:

- Include all major application components such as **User Authentication**, **Dashboard**, **Meal Planning**, **Nutrition Suggestions**, **Nutrition Tracking**, and **Profile Management**.
- Clearly distinguish the system architecture by showing the **React.js Frontend**, **Node.js & Express.js Backend**, and **MongoDB Database**.
- Indicate the infrastructure used during development, including **Local Development Environment**, **REST APIs**, and **MongoDB** for data storage.
- Show the interaction between the **Frontend**, **Backend**, and **Database** through secure API communication and **JWT Authentication**.
- Include the data storage components and illustrate how user information, meal plans, and nutrition records are securely stored in **MongoDB**.
- Represent the integration of the **Nutrition Suggestion Engine** used to generate personalized nutrition recommendations based on user inputs.

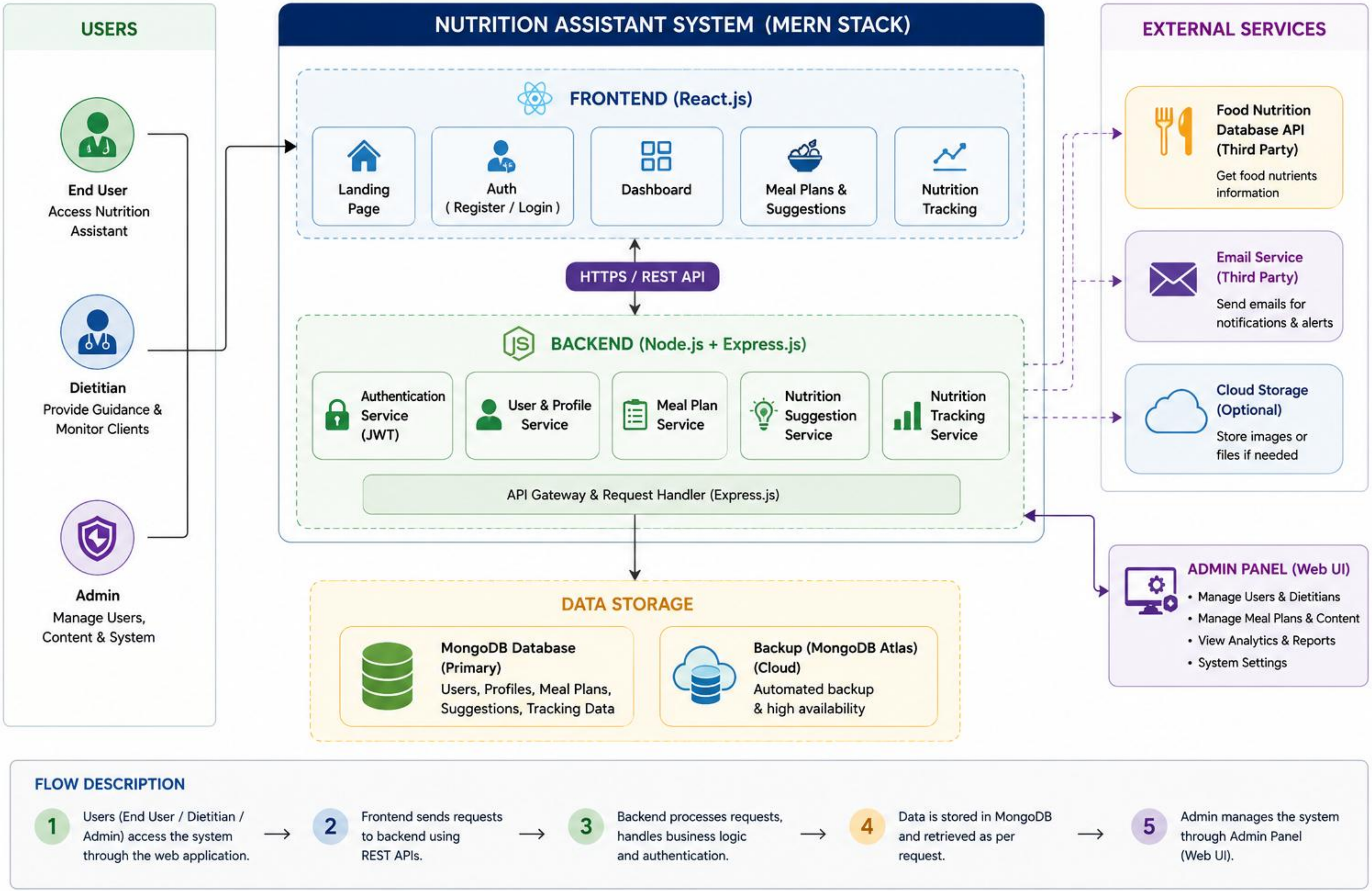
USERS
End User
Access Nutrition Assistant
Dietitian
Provide Guidance & Monitor Clients
Admin
Manage Users, Content & System
NUTRITION ASSISTANT SYSTEM (MERN STACK)
FRONTEND (React.js)
Landing Page
Auth (Register / Login)
Dashboard
Meal Plans & Suggestions
Nutrition Tracking
HTTPS / REST API
BACKEND (Node.js + Express.js)
Authentication Service (JWT)
User & Profile Service
Meal Plan Service
Nutrition Suggestion Service
Nutrition Tracking Service
API Gateway & Request Handler (Express.js)
EXTERNAL SERVICES
Food Nutrition Database API (Third Party)
Get food nutrients information
Email Service (Third Party)
Send emails for notifications & alerts
Cloud Storage (Optional)
Store images or files if needed
ADMIN PANEL (Web UI)
• Manage Users & Dietitians
• Manage Meal Plans & Content
• View Analytics & Reports
• System Settings
DATA STORAGE
MongoDB Database (Primary)
Users, Profiles, Meal Plans, Suggestions, Tracking Data
Backup (MongoDB Atlas) (Cloud)
Automated backup & high availability
FLOW DESCRIPTION
1 Users (End User / Dietitian / Admin) access the system through the web application.
2 Frontend sends requests to backend using REST APIs.
3 Backend processes requests, handles business logic and authentication.
4 Data is stored in MongoDB and retrieved as per request.
5 Admin manages the system through Admin Panel (Web UI).

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Responsive web application for users and administrators to interact with the system	React.js, HTML5, CSS3, JavaScript
2.	Application Logic-1	Handles user registration, login, and secure session management	Node.js, Express.js, JWT, bcrypt
3.	Application Logic-2	Creates and manages personalized meal plans based on user preferences	Node.js, Express.js
4.	Application Logic-3	Generates nutrition suggestions and dietary guidance	Express.js, Custom Nutrition Logic
5.	Database	Stores user accounts, meal plans, nutrition records, and suggestions	MongoDB, Mongoose
6.	Cloud Database	Cloud-hosted database for deployment and data backup	MongoDB Atlas
7.	File Storage	Stores profile images and uploaded assets (if available)	Local Storage / Public Folder
8.	External API-1	Connects frontend with backend services	Axios, REST APIs
9.	External API-2	Backend API testing during development	Postman
10.	Machine Learning Model	Not Applicable	Not Applicable
11.	Infrastructure (Server / Cloud)	Local development and deployment environment	Node.js Server, MongoDB Atlas

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source frameworks used for building the frontend, backend, database connectivity, and API communication.	React.js, Node.js, Express.js, MongoDB, Mongoose, Axios
2.	Security Implementations	Implements secure authentication, encrypted password storage, protected API routes, and role-based authorization.	JWT Authentication, bcrypt, Express Middleware
3.	Scalable Architecture	Three-layer MERN architecture with separate frontend, backend, and database, allowing future feature expansion.	MERN Stack (React.js, Node.js, Express.js, MongoDB)
4.	Availability	Web application is available through modern web browsers and supports cloud database connectivity for reliable access.	Node.js Server, MongoDB Atlas
5.	Performance	Optimized REST APIs, efficient database queries, and responsive frontend for faster application performance.	Axios, MongoDB Indexing, React.js Optimization

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d